TUPÃ

Da ideia ao hardware.

A linguagem esconde o hardware. O compilador faz o trabalho difícil.



Michel Lutegar · André Costa · Felipe · Guilherme Lopes

Linguagem Formal e Compiladores · IBMEC

O PROBLEMA

Escrever para hardware não deveria ser difícil

Hoje, uma ideia simples esbarra na complexidade do C++/Arduino:



TUPÃ (.tupa) — a mesma ideia

```
ao botao(1) {
    cartao = nfc.ler();

    se cartao.valido == verdadeiro {
        led.verde();
        esperar(1500);
    } senao {
        led.vermelho();
    }
}
```

O que já existe — e a lacuna

Flipper Zero

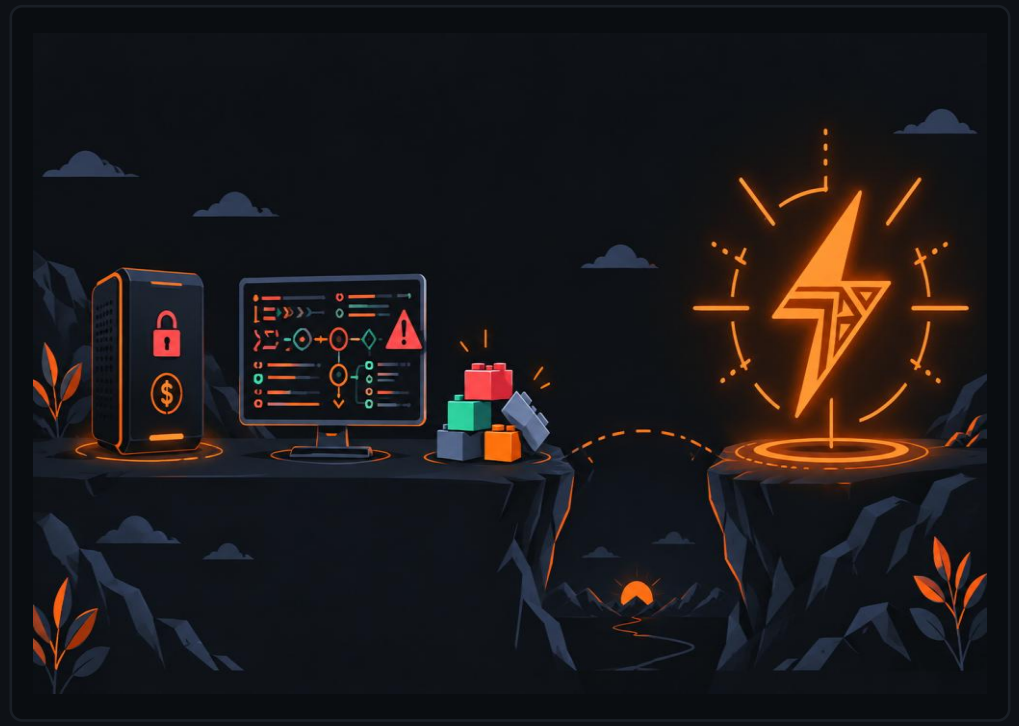
Hardware fechado e caro (~US$169). Inspira o conceito, mas não é linguagem nem foco em ensino.

Arduino IDE (C++)

Poderoso e maduro, mas exige saber C++, pinos e bibliotecas. Barreira alta para iniciantes.

MakeCode / Blockly

Blocos visuais para ensino, mas não geram Arduino nativo nem uma linguagem textual real.



A lacuna: uma linguagem textual em português, que compila para C++/Arduino nativo e roda no ESP32 — feita para makers e ensino.

A SOLUÇÃO

Uma linguagem em português + um compilador de verdade

Uma linguagem, não um framework

Sintaxe própria em português, com eventos de hardware de primeira classe: ao botao(1) { ... }.

Um compilador de verdade

Léxico, sintático, semântico e geração de código — as quatro fases, em Python.

Saída que roda no mundo real

Transpila para .ino, compila na Arduino CLI e grava no ESP32. Termina com um LED aceso.



O usuário escreve o QUE quer, não COMO o hardware faz.

Modelo educacional + open-source

Proposta de valor

Programar hardware sem saber C++ — em português, com um compilador de verdade.

Segmentos

Makers/hobbistas · escolas técnicas e universidades · entusiastas de segurança.

Canais

GitHub (open-source), comunidades maker, disciplinas de embarcados.

Sustentação futura

Cursos e oficinas · kits de hardware · suporte/consultoria educacional.



COMO FUNCIONA

O diferencial é o compilador

.tupa	Lexer	Parser	Semântica	Codegen	ESP32
texto	tokens	AST	tipos / hardware	C++ / .ino	grava



DEMO — MOSTRAR NA PLACA

Um controle de acesso por cartão



▶ DEMO AO VIVO

Aproximar o cartão e mostrar o LED acender. Backup: vídeo de 15 s.

controle.tupa — examples/controle_nfc.tupa

```
dispositivo controle {
    inteiro tentativas = 3;

    ao botao(1) {
        # le um cartao NFC
        cartao = nfc.ler();

        se cartao.valido == verdadeiro {
            led.verde();
            esperar(1500);
        } senao {
            led.vermelho();
        }
    }
}
```

A LINGUAGEM PROTEGE

Quando o código está errado, o compilador avisa

A análise semântica pega o erro ANTES do upload — mensagem clara, em português, e a lista do que existe. Tom didático, não punitivo.



Separa um "tradutor de texto" de um compilador de verdade.

SAÍDA DO COMPILADOR

```
$ tupa build controle.tupa

erro semantico - linha 9, coluna 9
    led.roxo();
        ^^^^
metodo inexistente no modulo 'led'

disponiveis: verde(), vermelho(),
             apagar()

compilacao interrompida: 1 erro
```

SOB O CAPÔ

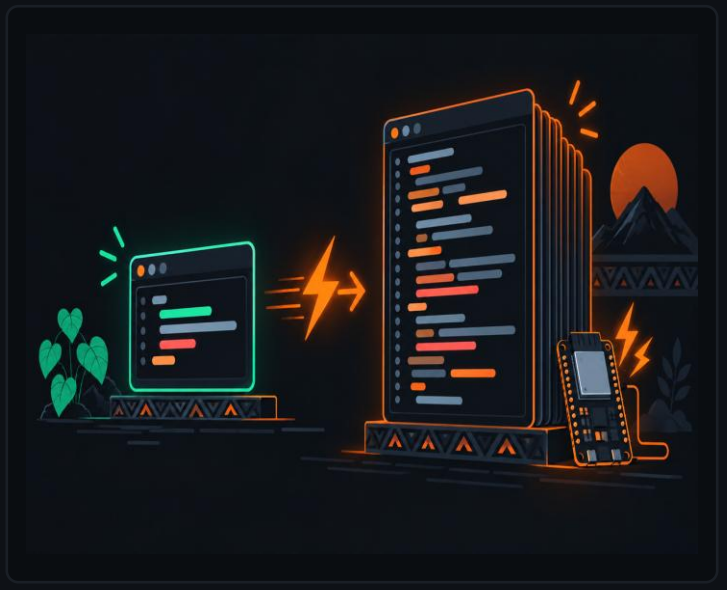
12 linhas de Tupã, 75 de C++

JOÃO ESCREVE — controle.tupa

- 12 linhas
- 0 includes
- 0 definições de pino
- 0 setup/loop
- Palavras em português

COMPILADOR ENTREGA — controle.ino

- 75 linhas (68 de código)
- Bibliotecas importadas
- Pinos configurados
- setup() e loop() prontos
- C++ válido p/ Arduino CLI



~6×

mais código em C++

0

linhas de hardware p/ o usuário

43

tokens na linguagem

PARA QUEM

Três pessoas, a mesma dor



João · o Maker

Estudante, curte eletrônica, não domina C++.

Ganho: escrever poucas linhas e ver funcionando.

Marina · a Professora

Ensina embarcados para iniciantes.

Ganho: usar a DSL como ferramenta didática.

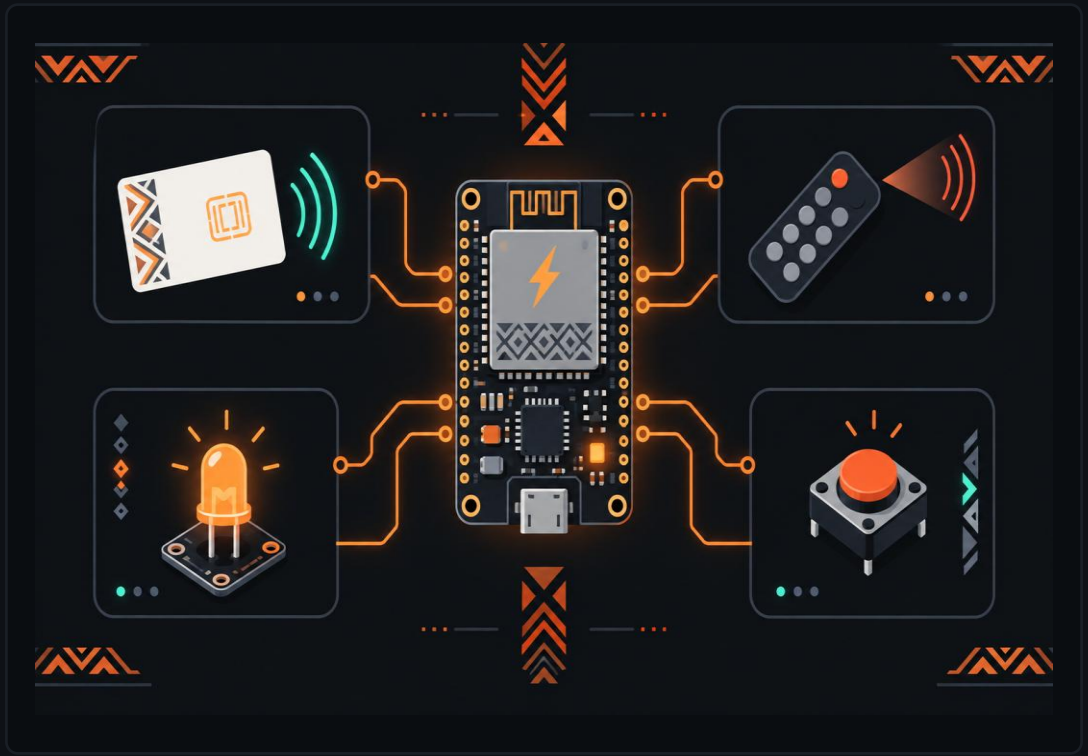
Caio · o Segurança

Testes autorizados de NFC/RFID e IR.

Ganho: scripts curtos e reutilizáveis.

Escopo enxuto, ecossistema maduro

MÓDULOS DO MVP (NFC · IR · LED · botões)



TECNOLOGIAS

Compilador	Python 3
Código gerado	C++ / Arduino (.ino)
Hardware	ESP32 DevKit v1
Toolchain	Arduino CLI / PlatformIO
Léxico	43 tokens
Extensão	.tupa

Linha do tempo do semestre

Set — 1ª entrega — Pitch

Out — Vídeo tutorial

Out — Artigo IEEE

Nov — Documentação GIT (léxico→codegen)

Nov — Demo na placa (ESP32)

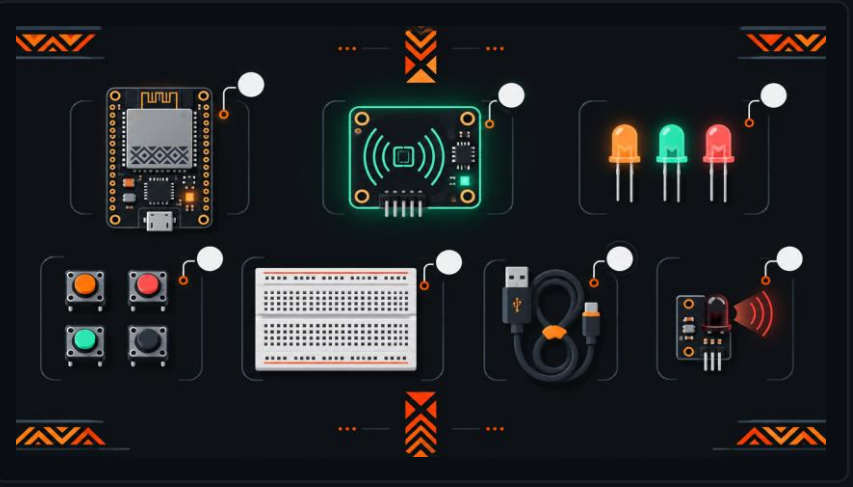


Custo do protótipo (estimado)

Componente	Custo
ESP32 DevKit v1	R$ 40
Leitor NFC/RFID MFRC522	R$ 15
Emissor + receptor IR	R$ 10
LEDs + resistores	R$ 5
Botões	R$ 5
Protoboard + jumpers	R$ 30
Cabo USB	R$ 15
TOTAL (hardware)	**≈ R$ 120**

SOFTWARE

R$ 0 — tudo open-source:
Python · Arduino CLI · PlatformIO · bibliotecas Arduino.



Onde estamos e para onde vamos

1 · Análise Léxica	✓ implementada
2 · Análise Sintática	✓ implementada
3 · Análise Semântica	✓ implementada
4 · Geração de Código	✓ implementada
5 · Integração ESP32	gravar na placa

POR QUE VIÁVEL

Compilador funcionando (4 fases) · 11 testes ok · exemplo canônico rodando · equipe de 4.

FUTURO (fora do MVP) — OTA por Wi-Fi



Próximas entregas: pitch → vídeo tutorial → artigo IEEE → documentação Git → demo na placa.

Obrigado.

Perguntas? A placa está aqui do lado.



Michel Lutegar · André Costa · Felipe · Guilherme Lopes

Linguagem Formal e Compiladores · IBMEC